

AI-Based Decision Support System for Industrial Processes

Design and Deployment of an LLM-Driven Analytical System

Public work sample based on a master’s thesis project in a large automotive manufacturing environment. All descriptions are intentionally generalized and safe for public sharing.

Context	Master’s thesis project focused on AI-supported bottleneck detection and recommendation for industrial operations.	Core stack	Python, Databricks, Spark, SQL, MLflow, vector search, REST APIs
Objective	Turn fragmented operational evidence into a faster, traceable decision-support workflow for engineers and managers.	Role fit	LLM application engineering, RAG, orchestration, data pipelines, testing, CI/CD-aware deployment, and production-readiness thinking

Problem Context

This work sample is based on a master’s thesis project on a decision-support system for bottleneck analysis in a complex manufacturing environment. The main challenge was not a lack of data; it was the effort required to turn scattered operational signals into a reliable explanation of where throughput was being constrained and what should be investigated first.

In practice, analysts had to switch between KPI views, downtime data, cycle-time signals, event histories, and written process knowledge. That made investigations slow, dependent on individual experience, and difficult to standardize. The goal of the project was to shorten that path by combining governed data access with LLM-based reasoning in a way that stayed traceable and useful for real operational decisions.

- The data landscape mixed structured sources such as KPIs, event data, and operational metrics with unstructured context such as playbooks, definitions, and analytical guidance.
- Bottleneck questions were rarely answerable with a single query; they typically required time-window comparison, evidence gathering, and interpretation across several signals.
- Any AI layer had to support faster, data-driven decision-making without bypassing access control, reliability, or the need for human review.

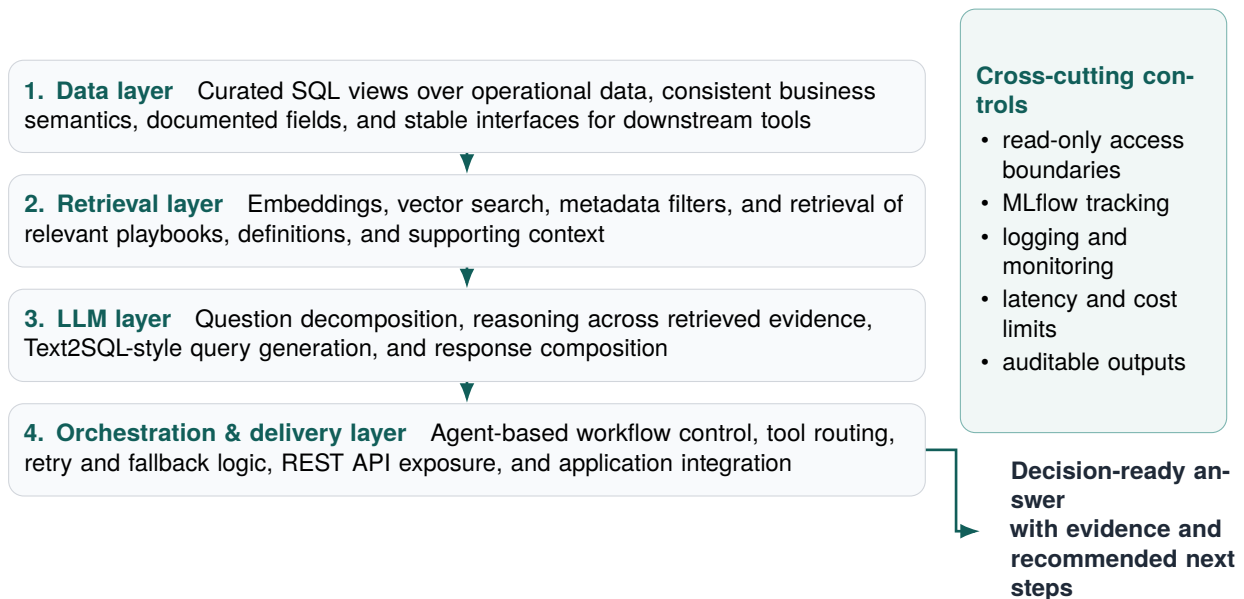
Design principle: the system separates exact data retrieval from generative reasoning. Structured facts come from curated SQL-accessible data, contextual guidance comes from retrieval, and the LLM is used for decomposition, synthesis, and recommendation rather than for guessing operational facts from memory.

Why this required more than classic BI

- Existing dashboards could show that a KPI changed, but they did not standardize how to investigate the cause across several data sources.
- The practical need was a guided workflow that could combine exact facts, operational context, and a clear explanation of what to check next.
- That made tool selection a system design problem: SQL for precise facts, retrieval for context, and LLM orchestration for multi-step reasoning and communication.

System Design & Architecture

The solution was designed as a modular pipeline so each responsibility could be evaluated and improved independently: data preparation, retrieval, reasoning, orchestration, and delivery. This is directly relevant to AI Engineering roles because the core work is not only prompt design; it is the design of a dependable system boundary around the model, its tools, and the surrounding data contracts.



- The architecture was intentionally modular so retrieval methods, model endpoints, and orchestration rules could be replaced without changing the full solution.
- Curated data interfaces reduced schema ambiguity and made LLM-driven access safer than exposing raw operational sources directly.
- The retrieval and reasoning layers were coupled through explicit workflow steps, which improved traceability, observability, and testability compared with a single-prompt chatbot design.

AI Approach

- **RAG:** Retrieval-augmented generation was used to ground outputs in governed data views and stored analytical guidance rather than in model recall.
- **Embeddings and vector search:** semantic retrieval helped map user questions to the right playbooks, definitions, and investigation context.
- **Prompt orchestration:** multi-step workflows decomposed a question into retrieval, query, comparison, and synthesis tasks.
- **Structured query generation:** KPI and time-window questions were translated into SQL with schema-aware prompts and constrained templates.

Trade-offs. Richer retrieval improved answer quality, but it added latency and tuning effort around chunking, metadata, and ranking. Agent-based decomposition improved complex investigations, but it also required guardrails to keep tool calls bounded, reliable, and explainable.

Integration & Data Handling

- The AI layer connected to data systems through curated interfaces instead of querying raw sources directly.
- Databricks, Spark, and SQL were used to prepare analytical views with stable naming, documented semantics, and reusable time-grain logic.
- REST APIs separated user-facing applications from orchestration and retrieval services, which made the solution easier to integrate into business systems and easier to evolve without changing every consumer at once.
- Schema context, metadata filters, and documented view semantics reduced prompt ambiguity and improved the reliability of structured query generation.
- Reliability depended on practical controls: input validation, timeout handling, empty-result fallbacks, and explicit error messages instead of silent failure.
- Access followed a read-only model aligned with existing permissions, which is essential when AI is embedded into operational workflows.

Key Contributions

- Designed the end-to-end system architecture across data preparation, retrieval, reasoning, orchestration, and delivery.
- Built Python, Spark, and SQL pipelines to expose heterogeneous operational data through stable analytical interfaces suitable for both BI and LLM workflows.
- Implemented multi-step LLM workflows that combined retrieval, Text2SQL-style access, and evidence synthesis instead of relying on a single prompt.
- Defined tool interfaces and service boundaries so retrieval, query execution, and response generation could be developed and observed as separate concerns.
- Defined evaluation and testing logic for answer usefulness, retrieval relevance, workflow behavior, and failure handling under incomplete or ambiguous inputs.
- Integrated experiment and configuration tracking with MLflow so prompts, workflow variants, and evaluation runs could be compared in a reproducible way.

Evaluation & Tool Selection

- Model and tool choice was treated as a task-level decision rather than a single platform choice. Exact KPI questions favored structured SQL access, contextual questions favored retrieval, and multi-step investigations used orchestration across both.
- Retrieval quality was evaluated through the relevance of returned playbooks and supporting context. Workflow quality was evaluated through usefulness, traceability, tool behavior, and graceful failure handling under imperfect inputs.
- MLflow was used to compare prompt variants, retrieval settings, and workflow versions against representative question sets, making it easier to reason about regressions, testing outcomes, and iteration quality.

Results & Impact

For public sharing, impact is described qualitatively rather than with internal business figures.

- The system reduced manual tool-switching between KPI views, context sources, and ad-hoc analysis steps.
- It improved decision support by standardizing how investigations were started, scoped, and explained.
- Retrieval quality and answer consistency increased through iterative tuning of metadata, prompts, and workflow structure.
- The project showed that generative AI is useful in industrial analytics when it is grounded in governed data access, clear service boundaries, and explicit workflow logic rather than autonomous control.

Production Readiness

- **Monitoring:** log retrieval hits, SQL execution outcomes, latency, tool paths, and user feedback so regressions are visible early.
- **Versioning:** track prompts, retrieval settings, workflow graphs, and model endpoints so behavior changes can be compared and rolled back deliberately.
- **CI/CD thinking:** treat prompt, workflow, and API changes as versioned software changes with controlled validation and deployment rather than direct production edits.
- **Cost awareness:** keep retrieval narrow, limit agent steps, and reserve larger models for reasoning-heavy stages rather than simple routing tasks.
- **Reliability:** include confidence checks, retry logic, empty-result handling, and clear human-readable fallbacks.
- **Scalability:** modular APIs, reusable orchestration patterns, and curated data contracts make the pattern easier to extend to additional processes or sites.
- **Safe deployment:** read-only access, auditable outputs, and explicit ownership of model behavior are prerequisites for enterprise adoption.

Why this work is relevant to AI Engineering roles: the project combined LLM application design, retrieval pipeline engineering, structured data access, testing and evaluation, API integration, and production controls around CI/CD, cost, latency, reliability, and observability. The result is closer to an engineered AI system than to a standalone chatbot prototype.